TD n° 3 et 4 : la classe ListOfStrings

Eric REMY
eric.remy@univ-amu.fr

Version du 24 février 2025

Cette planche d'exercices propose d'étudier l'implantation d'une classe ListOfStrings permettant de constituer une liste ordonnée de chaînes de caractères. Le but de la planche est de découvrir et d'apprendre à coder une *liste chaînée* : une *collection* qui a des propriétés différentes et complémentaires de celle du classique std::vector<std::string>.

Exercice 1

On va commencer par un exemple simple, sans créer de classe C++. Étudiez le code suivant ainsi que la sortie qu'il produit à l'écran.

```
1  struct Maillon {
2    int valeur;
3    Maillon *suivant;
4  };
5
6  int main()
7  {
8      Maillon *ptete = nullptr; // La liste ne contient pour l'instant aucun élément :
9                                // le pointeur « ptete » ne montre aucun premier maillon.
10     {
11         Maillon *pm1 = new Maillon; // On crée un premier maillon m1.
12         pm1->valeur  = 123; // On met la valeur 123 dedans.
13
14         Maillon *pm2 = new Maillon; // On crée un deuxième maillon m2.
15         pm2->valeur  = -234; // On met la valeur -234 dedans.
16
17         Maillon *pm3 = new Maillon; // On crée un troisième maillon m3.
18         pm3->valeur  = 543; // On met la valeur 543 dedans.
19
20         // On forme la liste en mettant les maillons dans l'ordre des valeurs croissantes.
21         ptete        = pm2; // Le pointeur de tête montre le premier maillon, celui qui est à l'adresse pm2.
22         pm2->suivant = pm1; // Le maillon qui suit m2 est celui qui est à l'adresse pm1.
23         pm1->suivant = pm3; // Le maillon qui suit m1 est celui qui est à l'adresse pm3.
24         pm3->suivant = nullptr; // Il n'y pas de maillon à la suite de m3.
25     } // À partir d'ici, les variables (pointeurs) pm1, pm2 et pm3 sont détruites mais pas les maillons
26       // sur lesquels elles pointaient !
27
28     for(
29         const Maillon* p=ptete; // En partant d'une variable p qui montre le premier maillon
30         p!=nullptr ; // Tant qu'on montre un maillon valable (≠ nullptr)
31         p = p->suivant // À la fin de chaque tour, on passe du maillon à son suivant.
32       )
33         std::cout << p->valeur << std::endl; // On affiche la valeur de ce maillon.
34
35     ... // Ici il faut libérer la mémoire allouée.
36 }
```

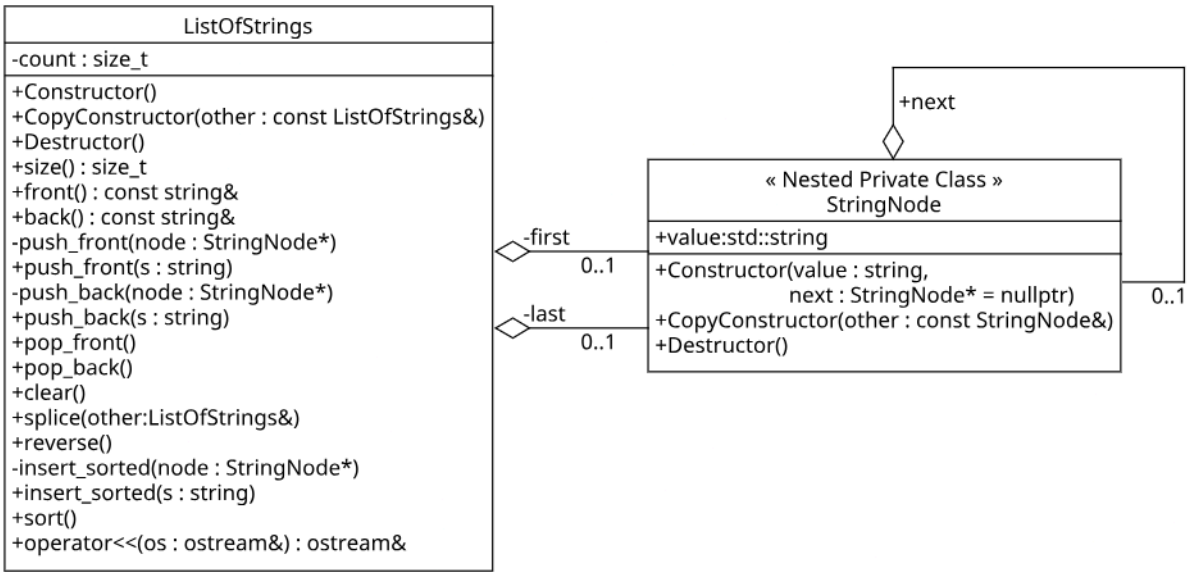
```
1  -234
2  123
3  543
```

1. Assurez-vous de comprendre chaque ligne du programme. En respectant la notation UML, dessinez le diagramme de classe, puis le diagramme d'objets correspondant à l'état de la liste à la ligne 30 de ce programme.
2. Les éléments de la liste sont-ils dans l'ordre où ils ont été créé ? Que faut-il faire pour ajouter un nouvel élément (au début, au milieu, à la fin) ? Pour en enlever un ? Que se passerait-il si on remplaçait la ligne 27 par « pm3->suivant = pm2; » ?
3. En vous inspirant du parcours de la liste qui sert à l'affichage des valeurs qu'elle contient (lignes 31-36), sauriez-vous comment modifier la fin du programme pour que les zones de mémoire allouées dynamiquement soient correctement libérées ?

4. Une telle structure de donnée est adaptée à un parcours dans un seul sens (du premier au dernier). Comment feriez-vous pour permettre de revenir en arrière et pas seulement avancer ?

Exercice 2

Considérez le diagramme de classe UML suivant qui montre comment implanter la classe ListOfStrings ainsi que sa sous-classe (privée) StringNode. La classe ListOfStrings reprend la structure de liste chaînée vue à l'exercice précédent en l'améliorant : non seulement elle contient un pointeur vers le premier maillon mais également un pointeur vers le dernier maillon (pour faciliter l'ajout en fin de liste), ainsi qu'un décompte des éléments actuellement présents dans la liste.



1. Quelles sont les données membres / attributs de la classe ListOfStrings ? De StringNode ? Sont-elles publiques ? Donnez les déclarations de ces deux classes.
2. Implantez le constructeur par défaut. Il doit créer une liste vide (qui ne contient aucun élément).
3. Implantez la fonction privée push_front(StringNode* node) qui insère en tête de liste une instance de StringNode fournie en paramètre. Puis, en vous servant de la fonction précédente, implantez la fonction push_front(std::string) qui ajoute au début de la liste la chaîne fournie en paramètre en allouant dynamiquement le maillon StringNode nécessaire à cela.
4. Sur le même principe, implantez la fonction privée push_back(StringNode* node) puis push_back(std::string) qui ajoutent au fin de liste.
5. Implantez la fonction membre front() de façon à ce qu'elle renvoie une référence constante à la chaîne contenue dans le premier élément de la liste, ou lance une exception de type std::out_of_range si la liste ne contient aucun élément. Faire de même pour back() pour le dernier élément de la liste.
6. Implantez les fonctions membres pop_front() et pop_back() qui suppriment respectivement le premier et le dernier élément de la liste. Le temps de calcul de pop_back() est-il constant ?
7. Implantez le constructeur de copie. Rappel : l'instance résultant de la copie doit être totalement indépendant de l'instance source de la copie.
8. Implantez la fonction splice() (en français, raccorder, mettre bout à bout) qui reçoit en paramètre une ListOfString et qui retire tous les éléments de cette liste pour les ajouter à la suite de ceux de la liste courante.
9. Implantez la fonction reverse() qui renverse l'ordre des éléments de la liste courante : le premier devient le dernier, le second devient l'avant-dernier, etc.
10. Implantez les deux fonctions insert_sorted(). Ces fonctions supposent que la liste est triée suivant l'ordre alphabétique de ses éléments. Le but est d'insérer le nouvel élément fourni à sa place dans la série ordonnée.
11. Implantez la méthode sort() qui trie tous les éléments d'une liste. La méthode suggérée consiste à détacher de la liste tous ses maillons (non-triés), la rendant de fait vide, puis à les insérer un par un à leur place légitime grâce à la fonction privée insert_sorted().

Remarque : l'implantation du destructeur, des méthodes size() et clear() ainsi que de l'opérateur d'injection sur un flux est à votre charge. Elles ne seront pas corrigées en TD car vous avez les éléments pour les réaliser par vous-même.